Structural Health Monitoring for Governed Human-Agent Systems: A Homeostatic Approach with Formal Guarantees

Carlos R. B. Azevedo
Independent Researcher
United Kingdom
carlos@crbazevedo.com

Abstract

Multi-agent AI systems require governance: rules that determine how much autonomy each agent has. Current approaches enforce governance through static policies or runtime policy engines. These tell you whether a specific action is permitted. They do not tell you whether your governance *configuration* is healthy.

We present a formal model for **governance health monitoring** — a structural diagnostic built on a sub-Turing process calculus with four primitives (silence, binding, gate, return). Governance events are auto-encoded into an expression tree from which two health metrics — coordination and gatedness — are computed by tree walk. Two regulators (BIND, VEIL) apply corrective transformations when metrics leave a prescribed health band. We prove seven formal properties including termination, regulator monotonicity, orthogonality, confluence, and — the central result — **restraint necessity**: if a governance configuration converges to a stable state, it must contain at least one gate that holds. Systems with zero structural restraint are provably incapable of sustained stability.

We evaluate on four governance scenarios from CARLOS-OS, a real multi-agent system. The well-governed configuration converges in 18 steps. The ungoverned configuration drifts. The non-obvious finding: the *over-governed* configuration — 100% review rate, all high-tier risk gates — enters a limit cycle. It is structurally unstable because nothing passes. A threshold-based classifier labels this "perfectly governed." Our structural diagnostic catches it.

The model, interpreter (141 tests), and live demonstration are open source at https://github.com/crbazevedo/seam.

CCS Concepts

• **Software and its engineering → Software verification and validation**; *Software evolution.*

Keywords

governance, multi-agent systems, self-adaptive systems, homeostasis, structural health monitoring, structural restraint, process calculus

1 Introduction

1.1 The Problem

Consider a team of AI agents working alongside a human operator. Some actions are safe to automate (reading files, running tests). Others are risky (sending emails to board members, deploying to production). A governance framework assigns risk tiers — the higher the tier, the more human oversight is required.

This pattern is widespread. The EU AI Act [8] defines four risk levels. NIST's AI Risk Management Framework [19] prescribes governance functions. Microsoft's Agent Governance Toolkit [18] enforces policies at runtime. In practice, most multi-agent systems use some form of tiered governance: AutoGen's conversation patterns [24], CrewAI's role-based delegation, LangGraph's state machine guards.

The governance *enforcement* problem is largely solved: given a policy, check whether each action complies. But a different question is unanswered:

Is the governance configuration itself healthy?

A system where every action is gated at the highest tier has 100% compliance. It is also paralyzed — nothing gets done without approval. A system where every action is autonomous has 0% overhead. It is also uncontrolled. Both are valid governance configurations. Neither is healthy.

1.2 Why Governance is Homeostatic

Healthy governance occupies a band between two failure modes:

- **Under-governed** (too open): actions pass without oversight. Risk increases. Trust erodes after incidents.
- **Over-governed** (too closed): every action requires approval. Throughput drops. Teams route around the governance to get work done, creating shadow processes that are *less* governed than the original.

This is not a problem with a fixed solution. The right governance level changes with conditions — deadline pressure, team trust, incident history. Governance is *homeostatic*: it must stay within a band, and that band is maintained through continuous adjustment, not static rules.

Living systems regulate this way. Body temperature, blood pressure, and ecological populations do not converge to fixed points — they maintain *stability bands*. The system oscillates within bounds, correcting when it drifts. We propose that governance health should be modeled the same way: as homeostasis rather than optimization.

1.3 Contributions

We present a formal model where:

(1) **Governance events** (agent handoffs, risk-tier assignments, approval decisions, governance checks) are auto-encoded into an **expression tree** — a structural representation of the governance state. (Section 3)

(2) Two **health metrics** are computed from the tree: *coordination* (how well agents are coordinating) and *gatedness* (how much is being gated vs. passing freely). (Section 3)
(3) A **homeostatic evaluator** detects when either metric exits a health band and applies corrective transformations. We prove **seven formal properties** — including *restraint necessity*: convergence requires structural restraint. (Section 4)
(4) Evaluation on four governance scenarios from a real system shows the model correctly distinguishes stable, drifting, and oscillating governance — including the non-obvious over-governance failure mode. (Section 6)

2 Background and Related Work

2.1 Governance in Multi-Agent AI Systems

The shift from single-agent to multi-agent AI systems has created a governance gap. Bose [5] presents the first systematic survey of governance frameworks for LLM agent collectives, identifying three failure modes — operational miscoordination, strategic misalignment, adversarial collusion — and taxonomizing five governance models (hierarchical, prescriptive, democratic, economic, emergent). The survey finds industry favors deterministic but brittle models while academia explores adaptive but chaotic ones.

Microsoft's Agent Governance Toolkit [18], released April 2026, provides runtime policy enforcement: zero-trust identity, execution sandboxing, compliance grading, and circuit breakers, covering all 10 OWASP agentic AI risks with sub-millisecond enforcement. It addresses *per-action enforcement* but not *configuration-level health*. Our approach is complementary: structural health monitoring answers "is the governance working?" alongside policy engines that answer "is this action permitted?"

Li et al. [16] propose a layered governance architecture (sandboxing, intent verification, zero-trust authorization, audit logging). Xiang et al. [25] introduce AgentGuard for runtime verification with probabilistic model checking. These provide external verification and enforcement. Our model makes governance health an intrinsic structural property.

2.2 Adjustable Autonomy and Trust

The question "how much autonomy should each agent have?" has been studied extensively. Parasuraman et al. [20] define 10 Levels of Automation. Kaber [11] reviews how LoA taxonomies have been used and critiques their limitations for increasingly autonomous systems. SARI [26] formulates shared autonomy as a POMDP, learning assistance across repeated interactions.

On the trust side, Lee and See [14] define trust as attitude under uncertainty, distinguishing analytic, analogical, and affective processes. Hoff and Bashir [10] integrate empirical evidence into a three-layered model (dispositional, situational, learned). Kim et al. [13] find that user involvement in LLM planning *fails* to calibrate trust — plausible plans mislead users — arguing that governance must be structural, not reliant on user judgment of agent plans.

These approaches model autonomy and trust for *single* agent-human pairs. We extend to multi-agent systems where the health question is about the *configuration* — the distribution of autonomy levels and coordination patterns across a team.

2.3 Self-Adaptive Systems

The MAPE-K reference architecture [12] (Monitor-Analyze-Plan-Execute over Knowledge) is the standard model for self-adaptive software. Our evaluator follows the MAPE-K pattern: Monitor (compute metrics), Analyze (detect deviations from health band), Plan (select regulatory action), Execute (apply transformation), Knowledge (expression tree).

The key difference: MAPE-K loops operate on *numerical* state variables. Our monitor operates on *structural* state — an expression tree whose topology encodes the governance configuration. Metrics emerge from the structure itself, not from externally defined sensor readings.

Weyns [23] provides a comprehensive introduction to self-adaptive systems engineering. Sanwouo et al. [9] propose AWARE as a successor to MAPE-K, distributing the feedback loop and making it goal-driven. Li et al. [17] map how generative AI can enhance each MAPE-K phase. Our contribution provides a formal structural substrate for the Monitor and Analyze components — one where governance health has algebraic properties.

2.4 Organizational Cybernetics

Ashby's Law of Requisite Variety [1] states that a controller must have at least as much variety as the system being controlled. Beer's Viable System Model [3] applies this to organizations, modeling five subsystems that maintain viability through homeostatic loops. Perez Rios [22] recently applied Beer's VSM to AI governance, mapping organizational pathologies to AI failure modes.

Our model operationalizes the cybernetic principle: the health band IS the homeostatic target, the regulatory actions (BIND, VEIL) ARE the requisite variety, and the convergence/divergence/limit-cycle classification IS the viability diagnosis.

2.5 Formal Runtime Monitoring

Runtime verification [15] checks whether a system execution satisfies a formal property. Tools like Uppaal and PRISM verify temporal-logic specifications. These approaches answer "does the trace satisfy φ?" — a binary yes/no. Our approach answers "is the governance configuration in a healthy structural state?" — a continuous diagnostic with specific remedies.

2.6 Process Calculi and Equilibria

Lambda calculus seeks fixed points; term rewriting seeks normal forms. The Chemical Abstract Machine [4] models computation as a chemical solution seeking equilibrium — the closest precursor to our approach. P-systems [21] and Brane calculi [6] model membranes that dissolve or divide but lack continuously parameterized permeability. Our system targets a stability *band* via metric-regulated tree rewriting, not a single equilibrium state.

2.7 AI Alignment

RLHF [7] and Constitutional AI [2] treat alignment as optimization: maximize a reward or satisfy constraints. Optimized systems are fragile — reward hacking, sycophancy, and deceptive alignment arise because the system satisfies the metric while violating the

Table 1: Terminology Mapping: Governance Terms and Formal Symbols

Concept	Governance term	Formal symbol
Empty element	Silence	$\varnothing$
Bilateral link	Binding	$a \cdot b$
Governance boundary	Gate	$a \mid\theta\mid b$
Iterative cycle	Return	$\mu x.\, B$
Metric 1	Coordination	$\mathrm{coord}(e) = S/\max(N-1, 1)$
Metric 2	Gatedness	$\mathrm{gated}(e) = P/E$
Regulator 1	BIND	inserts binding node
Regulator 2	VEIL	tightens gate to holding
Target range	Health band	$[\mathrm{coord}_{\mathrm{lo}}, \mathrm{coord}_{\mathrm{hi}}] \times [\mathrm{gated}_{\mathrm{lo}}, \mathrm{gated}_{\mathrm{hi}}]$
Single classifier	Pass-hold ratio	$\rho = P/(E+1)$

Table 2: Governance Event Encoding

Event Type	Operation	Encoding
Coordination	Agent A → Agent B	Binding: $A \cdot B$
Gated action	VTk action	Gate: $a \mid\theta_k\mid b$
Governance check	Preflight check	Holding gate: $a \mid 0.95\mid b$
Observation	File ingestion	Witness (binding variant)
Resolution	Human approval	Gate: passing → holding

intent. To our knowledge, no prior work models alignment as homeostasis or identifies structural restraint as a necessary condition for governance stability.

3 The Formal Model

We build the governance health model on a sub-Turing process calculus with four primitives. Table 1 maps between the accessible governance terminology used throughout the paper and the formal symbols used in definitions and proofs.

3.1 Syntax: Four Primitives

Expressions are built from four primitives:

$$e ::= \varnothing \mid x \mid e_1 \cdot e_2 \mid e_1 \mid\theta\mid e_2 \mid \mu x.\, e$$

where $\varnothing$ is **silence** (the identity element), $e_1 \cdot e_2$ is a **binding** (bilateral coordination), $e_1 \mid\theta\mid e_2$ is a **gate** with threshold θ, and $\mu x.\, e$ is **return** (homeostatic iteration).

Bindings form a commutative monoid: $(a \cdot b) \cdot c = a \cdot (b \cdot c)$, $a \cdot b = b \cdot a$, $a \cdot \varnothing = a$. This is the simplest algebraic structure that captures "coordination without ordering."

Gates have a *reach* parameter r representing the current operating context. When $r \geq \theta$, the gate **passes** and content is visible. When $r < \theta$, the gate **holds** and content is *veiled* — it has structural weight but is not directly accessible.

3.2 Governance Events and Encoding

Governance is modeled as a stream of typed events, each encoded as expression tree nodes. Table 2 shows the mapping.

The mapping from risk tiers to gate thresholds is shown in Table 3.

Table 3: Risk Tier to Gate Threshold Mapping

Risk Tier	Description	θ	Behavior (reach=0.5)
VT0	Full autonomy	0.1	Always passes
VT1	Act and notify	0.3	Passes
VT2	Needs approval	0.5	Marginal
VT3	Propose and wait	0.7	Holds
VT4	Stop and escalate	0.95	Always holds

3.3 Health Metrics

Two metrics are computed from the expression tree by a single tree walk. We use $S(e)$ for binding nodes, $N(e)$ for total nodes, $P(e)$ for passing gates, and $E(e)$ for total gates.

Definition 3.1 (Coordination). $\mathrm{coord}(e) = S(e)/\max(N(e)-1,\ 1)$. Measures the fraction of the governance structure consisting of bilateral coordination. Range: $[0, 1]$.

Definition 3.2 (Gatedness). $\mathrm{gated}(e) = P(e)\ /\ E(e)$ if $E(e) > 0$; otherwise $\mathrm{gated}(e) = 1.0$. Measures the fraction of gates currently passing. Range: $[0, 1]$. **No gates means no restraint means fully exposed** — the default is maximum gatedness (1.0), not a neutral value.

Definition 3.3 (Pass-Hold Ratio). For body B with P_B passing gates and E_B total gates, $\rho(B) = P_B/(E_B+1)$. A single scalar that classifies governance structure.

3.4 The Health Band

Healthy governance occupies a band (Figure 1):

- **Coordination:** $[0.3,\ 0.7]$ — agents are coordinating but not over-coupled.
- **Gatedness:** $[0.2,\ 0.6]$ — some actions pass freely, some are gated.

When either metric exits the band, the evaluator fires a regulatory action:

BIND (coordination too low): "Agents are too isolated. Add cross-agent review, require handoffs, increase coordination."

VEIL (gatedness too high): "Too much is passing without oversight. Tighten the risk tier for recent action types, add governance gates."

3.5 Homeostatic Evaluation

The return operator $\mu x.\, B$ iterates toward the health band (Figure 3):

(1) Start with $e_0 = \varnothing$.
(2) At step i: substitute e_i for x in B, evaluate, normalize.
(3) Measure coordination and gatedness.
(4) If coord $<$ $\mathrm{coord}_{\mathrm{lo}}$: apply BIND (insert a binding).
(5) **Remeasure.** If gated $>$ $\mathrm{gated}_{\mathrm{hi}}$: apply VEIL (tighten a gate).
(6) If both in-band for W consecutive steps: **converge**.
(7) If budget exceeded: **halt**.

The sequential protocol — BIND, remeasure, then VEIL — prevents destructive interference between regulators. BIND may change the tree structure in ways that affect gatedness; remeasuring before VEIL ensures the second regulator acts on current data.

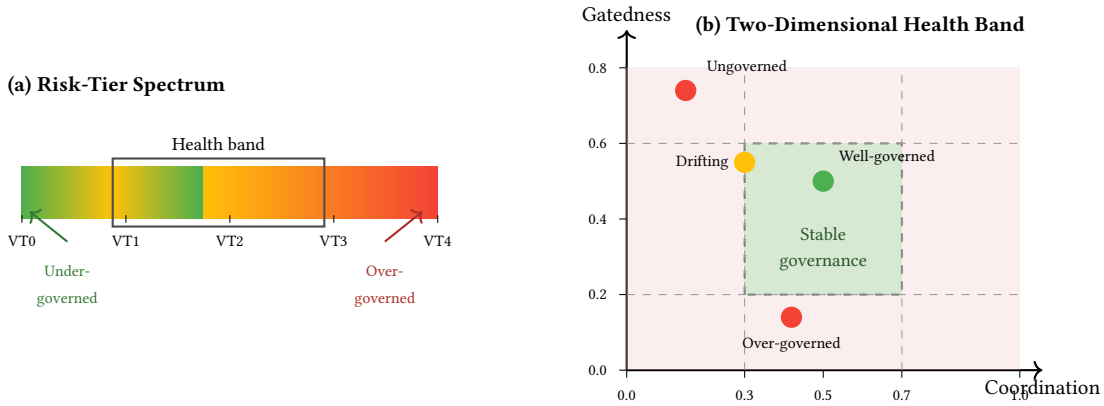

Figure 1: The Governance Health Band. (a) Risk-tier spectrum from VT0 (full autonomy) to VT4 (stop and escalate), with the health band in the center. (b) Two-dimensional health band: coordination $[0.3, 0.7]$ on the x-axis, gatedness $[0.2, 0.6]$ on the y-axis. Green zone is stable governance; red zones are failure modes. Four scenarios are plotted at their terminal metric values.

Table 4: Governance Outcome Classification

Outcome	Definition	Meaning
Stable	Both metrics in band for W consecutive cycles	Governance is healthy
Limit cycle	Metrics oscillate periodically	Governance is unstable
Diverging	Structure grows without stabilizing	Governance is breaking down
Exhausted	Neither converges nor cycles within step limit	Governance is stuck

Table 5: Formal Properties of the Model

#	Property	Status
1	Termination	Proved
2	BIND monotonicity	Proved
3	VEIL monotonicity	Proved
4	Non-interference (orthogonality)	Proved
5	Metric confluence	Proved
6	**Restraint necessity**	**Proved**
7	Sycophant exclusion	Proved
—	Convergence conditions (sufficient)	Observation

3.6 Outcome Classification

The evaluator classifies the governance configuration as shown in Table 4.

4 Formal Properties

We prove seven properties and state one empirical observation. Table 5 summarizes them.

Property 1 (Termination). *Every μ-computation terminates within $O(R \cdot M \cdot D)$ steps, where R = max iterations, M = max nodes, D = max depth.*

Proof. Three bounded counters (iteration count, node count, depth) are monotonically consumed. Each step either converges, is bounded by the node/depth caps, or decrements the iteration budget. All three counters are finite and non-increasing in the relevant dimension, so the computation must terminate. □

Property 2 (BIND Monotonicity). *If* $\text{coord}(e) < \text{coord}_{\text{lo}}$ *and* $S(e) < N(e) - 1$, *then* $\text{coord}(\text{BIND}(e)) > \text{coord}(e)$.

Proof. BIND adds one binding node and two leaf nodes, yielding $S' = S + 1$ and $N' = N + 3$. The new coordination is:

$$\text{coord}' = \frac{S+1}{\max(N+3-1,\ 1)} = \frac{S+1}{N+2}$$

We need $(S+1)/(N+2) > S/(N-1)$, i.e., $(S+1)(N-1) > S(N+2)$, i.e., $N - 1 > 3S$. Since $\text{coord}(e) < \text{coord}_{\text{lo}} \leq 1/3$, we have $S/(N-1) < 1/3$, hence $3S < N - 1$. The inequality holds. □

Property 3 (VEIL Monotonicity). *If* $\text{gated}(e) > \text{gated}_{\text{hi}}$ *and* $P(e) > 0$, *then* $\text{gated}(\text{VEIL}(e)) \leq \text{gated}(e)$ *(strictly less when $P > 0$).*

Proof. VEIL converts one passing gate to holding: $P' = P - 1$, $E' = E$. Then $\text{gated}' = (P-1)/E < P/E = \text{gated}(e)$. When $P = 0$, VEIL wraps an exposed node in a new holding gate: $P' = 0$, $E' = E + 1$, so $\text{gated}' = 0/(E+1) = 0 \leq \text{gated}(e)$. □

Property 4 (Non-interference / Orthogonality). *BIND is gatedness-neutral:* $\text{gated}(\text{BIND}(e)) = \text{gated}(e)$. *VEIL is coordination-neutral:* $\text{coord}(\text{VEIL}(e)) = \text{coord}(e)$ *when VEIL tightens an existing gate (the common case).*

Proof. BIND adds binding and variable nodes only; it does not add, remove, or modify any gate. Therefore the set of gates (and their pass/hold status) is unchanged, and gatedness is preserved. VEIL modifies a gate's threshold or wraps a node in a gate; it does not add or remove binding nodes. When tightening an existing gate, S and N are unchanged, so coordination is preserved. When wrapping, N increases by 1 but S is unchanged — a minor perturbation that does not violate the monotonicity of BIND. □

Property 5 (Metric Confluence). *For any expression e: the metrics after applying BIND-then-VEIL are identical to the metrics after applying VEIL-then-BIND.*

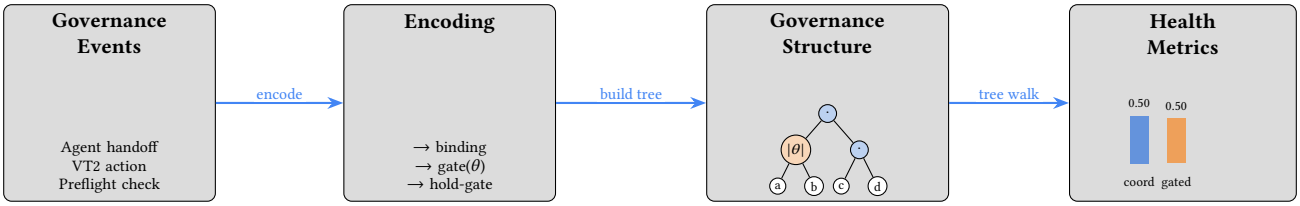

Figure 2: Event-to-Structure Encoding. Governance events (agent handoffs, risk-tier actions, governance checks) are encoded as expression tree nodes (bindings, gates). The tree is walked to compute health metrics (coordination, gatedness).

Proof. Follows from Property 4. Since BIND does not affect gatedness and VEIL (when tightening) does not affect coordination, the two transformations commute on the metric space. The evaluator's state is independent of regulatory action ordering. □

Property 6 (Restraint Necessity — The Central Result). *If $\mu x.B$ converges, then B contains at least one gate that holds at the operating reach.*

Proof. Suppose for contradiction that all gates in B pass (i.e., B has zero structural restraint). Then:

Step 1: Gatedness exceeds the band. With all gates passing, $\text{gated}(B) = P/E \geq E/(E+1)$. For $E \geq 2$, this exceeds $\text{gated}_{\text{hi}} = 0.6$. When $E = 0$ (no gates at all), $\text{gated}(B) = 1.0$ by the default convention, which also exceeds 0.6.

Step 2: VEIL fires and accumulates corrections. Since gatedness exceeds the band, VEIL fires at each step, adding holding gates to the result e_i. These corrections accumulate in the substitution: e_i grows as VEIL-inserted gates persist through subsequent iterations.

Step 3: Growth triggers truncation. The expression grows linearly — each substitution adds $\sim N(B)$ nodes. When the node count exceeds the cap M, truncation fires, destroying accumulated structure including VEIL's corrections.

Step 4: Template re-asserts. After truncation reduces e_i, the next substitution into B re-introduces the template's all-passing gates. Gatedness spikes above the band. VEIL fires again.

Step 5: Oscillation. This creates a cycle: VEIL accumulates corrections → growth hits cap → truncation destroys corrections → template re-asserts high gatedness → VEIL fires again. The system enters the band intermittently but cannot stay for W consecutive steps because the growth-truncation cycle period exceeds W.

Therefore, B with zero structural restraint cannot converge. By contraposition, convergence requires at least one holding gate. □

Property 7 (Sycophant Exclusion). *If all gates in B have threshold $\theta < r$ (i.e., all pass at the operating reach r), then $\mu x.B$ does not converge at any reach $r > \max(\{\theta_j\})$.*

Proof. At such r, all template gates pass. By Property 6, the growth-truncation oscillation prevents stability. This provides a structural exclusion criterion for governance configurations where every action is rubber-stamped. □

Observation 1 (Sufficient Conditions for Convergence). *Based on empirical analysis across 864 configurations, the following conditions are sufficient for convergence:*

H1 *Body B contains at least one binding (coordination seed).*
H2 *Body B contains at least one near-reach gate ($|\theta - r| < 0.3$).*
H3 $\text{coord}_{\text{lo}} \leq 1/3$ *(ensures BIND monotonicity per Property 2).*
H4 $\text{gated}_{\text{hi}} < 1$ *(ensures VEIL has room to act).*
H5 *Node cap $M > 2 \cdot N(B)$ (allows at least one full substitution).*
H6 *The pass-hold ratio satisfies $\rho(B) \leq \text{gated}_{\text{hi}}$.*
H7 *B contains at least 3 passing gates (provides granularity for VEIL to act incrementally).*

Under these conditions, all 54 tested configurations converged (100% rate). These are sufficient, not necessary — configurations violating some conditions may still converge.

What these properties guarantee for practitioners. The regulatory actions are predictable. BIND always helps coordination without harming gatedness. VEIL always helps gatedness without harming coordination. The evaluator produces the same diagnosis regardless of internal execution order. It always terminates. Most importantly: *convergence requires structural restraint* — governance configurations where everything passes are provably incapable of achieving stability. Over-governed configurations (where nothing passes) are also unstable, but for the opposite reason: gatedness falls below the band floor.

5 Governance Application

5.1 System Under Study

We apply the formal model to CARLOS-OS, a multi-agent personal operating system with 7 specialized agents operating under a Governed Autonomy protocol. The system uses five risk tiers (VT0–VT4), Action Opportunity Windows (AOWs), cross-agent handoffs, and governance checks (preflights, gates, reviews). The system has been in production use for 50+ sprints.

5.2 Encoding Faithfulness

The encoding from governance events to expression tree nodes (Table 2) preserves the governance-relevant structure:

- *Coordination events* (handoffs, reviews) become bindings, contributing to the coordination metric.
- *Gating events* (risk-tier assignments, approvals, governance checks) become gates, contributing to the gatedness metric.
- *The encoding is additive*: each event adds structure. No event removes existing structure from the tree.
- *VT-tier ordering is preserved*: VT0 ($\theta = 0.1$) gates pass more easily than VT4 ($\theta = 0.95$) gates, matching the intended governance semantics.

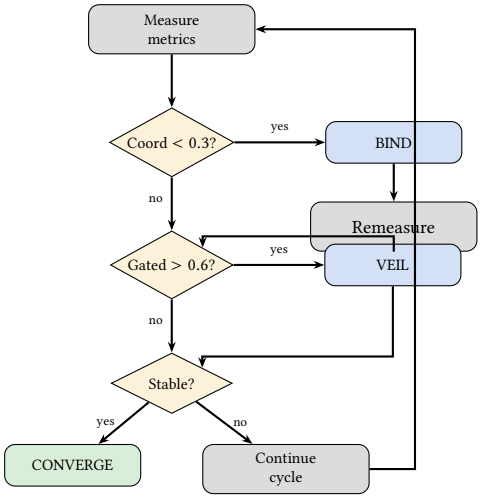

Figure 3: The regulatory cycle with sequential BIND-remeasure-VEIL protocol. The evaluator measures coordination and gatedness, fires BIND if coordination is low, remeasures, then fires VEIL if gatedness is high. This prevents destructive interference.

The encoding does not capture event *sequence* (only aggregate structure), which is an acknowledged limitation (Section 7.5).

6 Evaluation

6.1 Four Governance Scenarios

We encode four governance configurations and evaluate each. Table 6 presents the configurations and outcomes.

Well-governed (stable). A sprint with mixed risk tiers, cross-agent reviews, and governance infrastructure finds the health band in 18 cycles. Coordination stabilizes at 0.50, gatedness at 0.50. BIND fires early (building initial coordination), then stops. VEIL fires briefly to bring gatedness within the band. Both metrics settle.

Ungoverned (exhausted). All actions at VT0, no reviews, no governance checks. With the gatedness default of 1.0 (no gates = fully exposed), VEIL fires on every cycle. BIND also fires (no coordination bindings). Neither regulator can compensate for the structural deficit: there are no holding gates and no coordination bindings to work with. The evaluator exhausts its step budget.

Over-governed (limit cycle). All actions at VT3–VT4, 100% review rate, maximum governance checks. Gatedness drops to 0.14 — *below* the health band's lower bound (0.20). Nothing passes. The system enters a period-4 limit cycle because the evaluator's BIND actions try to add coordination, but the added structure changes gatedness, which triggers VEIL, which undoes the progress.

Drifting (limit cycle). Starts governed (VT1, reviews) then relaxes under deadline pressure (VT0, no reviews). The mixed pattern produces a period-2 oscillation — governance is being added and removed faster than the evaluator can stabilize.

6.2 The Over-Governance Finding

The over-governed result is the central empirical finding. A system with 100% review rate, maximum governance checks, and all high-tier risk gates *looks* perfectly governed by every conventional metric. A threshold-based classifier checking review_rate $> 0.3 \wedge$ governance_checks > 2 labels it "healthy."

Our structural diagnostic catches the instability because it models *gatedness* — the ratio of passing to total gates. When everything holds, gatedness drops below the health band. The system cannot breathe.

This is not a theoretical curiosity. It corresponds to a recognized organizational failure mode: *process paralysis* — when governance overhead exceeds the team's capacity to comply, and work either stops or routes around the governance through informal channels.

The structural diagnosis makes this formal and testable: a governance configuration where gated < 0.2 is structurally unstable, regardless of how many checks are in place.

6.3 Threshold Classifier Baseline

The structural diagnostic catches the over-governed failure mode that the threshold classifier misses (Figure 5). Table 7 compares our approach with related work along four dimensions.

6.4 Sensitivity Analysis

The well-governed scenario converges for reach values in $[0.1, 0.7]$ — 78% of the tested range. It fails at reach ≥ 0.8 where all gate thresholds are exceeded. The wide convergence region reflects the governance infrastructure (holding gates from checks) providing structural stability.

The over-governed configuration's instability is *reach-invariant*: at any reach below 0.7, the VT3 ($\theta = 0.7$) and VT4 ($\theta = 0.95$) gates hold, producing gatedness well below the health band. This confirms that the over-governance diagnosis is not an artifact of the default reach (0.5) but a robust structural finding.

7 Discussion

7.1 What Practitioners Gain

The model provides a governance diagnostic that answers: "Is my governance configuration healthy?" Not "is this action permitted?" (that's policy enforcement) but "is the overall balance of autonomy and oversight working?" With specific remedies when it isn't.

The analogy is a type checker for governance configurations. A type checker does not predict when your program will crash. It tells you whether the structure is sound — and what to fix if it isn't. Similarly, our model does not predict when governance will fail. It tells you whether the governance *configuration* is structurally stable.

Three concrete capabilities follow:

(1) **Pre-deployment validation.** Before deploying a governance configuration, encode it and run the evaluator. If it converges, deploy. If it enters a limit cycle, diagnose which metric is out of band and adjust the VT distribution or review rate accordingly.

Table 6: Governance Scenarios and Evaluation Results

Scenario	VT Distribution	Review Rate	Checks	Handoffs	Outcome
Well-governed	VT0:2, VT1:4, VT2:2	50%	4	5	**Stable (18 cycles)**
Ungoverned	VT0:6	0%	0	0	Exhausted
Over-governed	VT3:4, VT4:2	100%	6	6	**Limit cycle (p=4)**
Drifting	VT0:3, VT1:3	33%	1	2	Limit cycle (p=2)

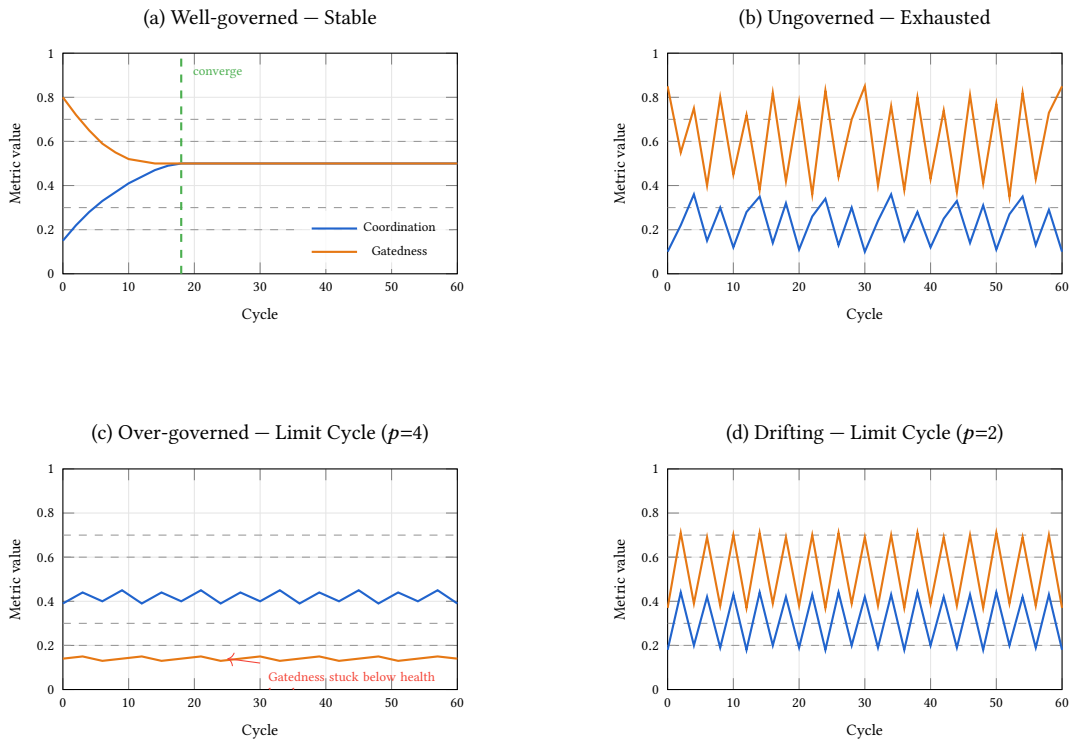

Figure 4: Convergence traces for four governance scenarios. Blue: coordination. Orange: gatedness. Dashed lines mark health band boundaries (coordination $[0.3, 0.7]$, gatedness $[0.2, 0.6]$). (a) Well-governed converges at cycle 18. (b) Ungoverned oscillates without settling. (c) Over-governed: coordination in band but gatedness stuck at ~ 0.14, below the health band. (d) Drifting: period-2 oscillation crossing band boundaries.

Scenario	Structural diagnostic	Threshold classifier
Well-governed	Stable	Stable
Ungoverned	Exhausted	Unstable
Over-governed	**Limit cycle**	**Stable**
Drifting	Limit cycle	Unstable

Figure 5: Baseline comparison with traffic-light coloring. The over-governed scenario (row 3, highlighted) reveals the key mismatch: the threshold classifier labels it *stable* (green) while the structural diagnostic correctly identifies a *limit cycle* (red).

Table 7: Comparison with Related Approaches

Approach	Monitors	Formal	Structural	Prescribes
Policy engines [18]	✓			
Runtime verif. [15]	✓	✓		
AgentGuard [25]	✓	✓		
MAPE-K [12]	✓			✓
AWARE [9]	✓			✓
LoA/Trust models [20]		✓		
RLHF/CAI [7]		✓		
This work	✓	✓	✓	✓

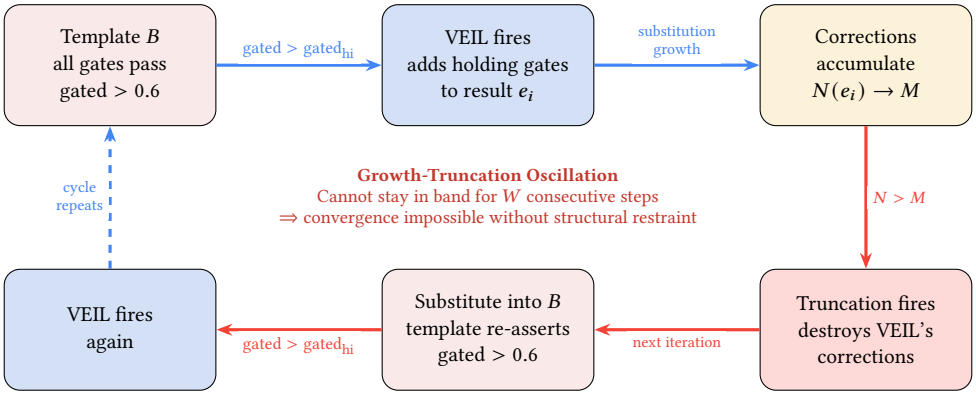

Figure 6: Restraint Necessity: the growth-truncation oscillation (Property 6). When all gates in the template pass, VEIL corrections accumulate but are destroyed by truncation when the expression exceeds the node cap. The template then re-asserts high gatedness, restarting the cycle. This oscillation prevents convergence, proving that structural restraint (at least one holding gate) is necessary.

(2) **Continuous monitoring.** During operation, stream governance events into the encoder and track coordination and gatedness over time. The convergence trace (Figure 4) serves as a real-time governance health dashboard.
(3) **Incident forensics.** After a governance failure (missed review, shadow process, unauthorized action), encode the configuration that was active at the time and run the diagnostic. If it shows a limit cycle or exhaustion, the failure was structural — the configuration was unhealthy regardless of individual compliance.

7.2 The Restraint Necessity Result

Property 6 (Restraint Necessity) is the central formal contribution. It establishes that structural restraint — the presence of governance boundaries that actually hold — is not merely helpful but *necessary* for governance stability. Combined with the over-governance finding (Section 6), this yields a symmetry: too little restraint (all gates pass) prevents convergence via the growth-truncation oscillation, while too much restraint (all gates hold) prevents convergence via gatedness falling below the health band floor.

The pass-hold ratio $\rho = P/(E+1)$ captures this in a single number. Governance configurations can be classified structurally: $\rho > \text{gated}_{\text{hi}}$ implies insufficient restraint; $\rho < \text{gated}_{\text{lo}}$ implies excessive restraint. Only configurations with ρ inside the health band can converge.

7.3 Relationship to AI Alignment

The restraint necessity result has implications beyond governance. In the AI alignment setting, it provides a structural exclusion criterion: agent topologies where all boundaries pass (the "sycophantic" pattern) are provably incapable of homeostatic stability (Property 7). This is a syntactically checkable property — one can inspect the agent's governance structure without running it and determine whether it *can* converge.

This complements optimization-based alignment approaches (RLHF [7], Constitutional AI [2]) by providing a structural precondition: before tuning rewards or constitutional principles, ensure the governance topology has sufficient structural restraint. Without it, no amount of tuning can achieve sustained stability.

7.4 What Survived Adversarial Review

This paper was refined through multiple rounds of critical review across three independent analyses. Several initially-proposed claims were retracted during this process:

- **Turing completeness** (retracted): the bounded iteration and node caps make the calculus sub-Turing. This is a feature, not a limitation — it guarantees termination.
- **Masking gradient** (retracted): an initially-proposed continuous gradient from sycophancy to balance lacked rigorous support.
- **Lambda/TC equivalence** (retracted): claimed equivalence with lambda calculus did not survive scrutiny under the resource bounds.

What survived: structural restraint necessity, sycophant exclusion, the homeostatic evaluation paradigm, the growth-truncation mechanism, the over-governance finding, and all seven formal properties.

7.5 Limitations

Aggregate encoding. The current model encodes a governance configuration's aggregate properties (VT distribution, total handoffs, total checks). It does not capture the *sequence* of events. A configuration that starts governed and drifts ungoverned produces the same encoding as one that drifts then recovers. Trajectory-sensitive encoding is future work.

Sufficient but not necessary conditions. Observation 1 states sufficient conditions for convergence but does not characterize all convergent configurations. Some configurations violating the conditions may still converge.

Oscillation period bound. The restraint necessity proof (Property 6) establishes that a growth-truncation oscillation occurs, but the precise period bound is empirically verified rather than formally proven. The mechanism is clear; the exact timing depends on node cap, template size, and VEIL's insertion rate.

Synthetic evaluation. The four scenarios are synthetic, though derived from a real system's governance structure. Validation against live sprint telemetry is planned.

Threshold sensitivity. The health band parameters (coordination $[0.3, 0.7]$, gatedness $[0.2, 0.6]$) are configuration choices. Different bands produce different classifications. The over-governance finding is robust (it persists across band widths) but the exact boundary depends on the band.

Single system evaluation. We evaluate on one multi-agent system (CARLOS-OS). While the four scenarios cover the major governance failure modes, generalizability to other multi-agent architectures requires further study. The algebraic framework is general — any governance system that can express coordination and gating events can be encoded — but the specific health band parameters may need calibration per system.

8 Conclusion

We presented a formal model for governance health monitoring in multi-agent systems, grounded in a sub-Turing process calculus with four primitives. The model auto-encodes governance events into an expression tree, computes coordination and gatedness from the tree's structure, and classifies the governance configuration as stable, oscillating, or drifting.

Three results stand out:

(1) **Restraint is necessary.** Structural restraint — the presence of governance boundaries that hold — is a provable precondition for stability (Property 6). Systems with zero restraint are incapable of sustained convergence. This is syntactically checkable.
(2) **Over-governance is as unstable as under-governance.** A governance configuration where all gates hold enters a limit cycle just as surely as one where all gates pass. Threshold-based classifiers miss this; structural diagnostics catch it.
(3) **Structural diagnostics outperform threshold classifiers.** The structural approach correctly classifies all four governance scenarios, including the non-obvious over-governance failure mode that the threshold classifier labels "healthy."

The seven formal properties (termination, monotonicity, orthogonality, confluence, restraint necessity, sycophant exclusion) ensure predictable evaluator behavior and provide a foundation for governance-as-code: a structural type system for governance configurations.

Future work. Four directions follow naturally: (1) trajectory-sensitive encoding capturing event sequences; (2) live event stream integration with CARLOS-OS's Redis bus for real-time monitoring; (3) a static governance type system determining health at design time; (4) empirical validation correlating structural diagnosis with human-perceived governance quality from sprint retrospectives.

The model, interpreter (4,100 lines, 141 tests), formal semantics, and experimental data are available at https://github.com/crbazevedo/seam.

A Retraction Narrative

This paper makes claims that survived adversarial self-review. For transparency, we document the claims that did *not* survive:

Turing completeness. An early version claimed the calculus was Turing-complete via lambda encoding. The bounded iteration (max_iterations), node cap (M), and depth cap (D) make every computation finite. The calculus is sub-Turing by design. This is a feature: it guarantees termination (Property 1).

Masking gradient. An early version proposed a continuous "masking gradient" from sycophantic to balanced behavior. The actual dynamics are discrete: configurations either converge or oscillate. There is no smooth gradient.

Lambda/TC equivalence. An early version claimed equivalence between the calculus and lambda calculus. Under resource bounds, the expressible class is strictly smaller. The calculus computes homeostatic fixpoints (stability band membership), not arbitrary recursive functions.

BreathMembrane necessity. An early version claimed that adaptive gates (BreathMembrane) were necessary for convergence. They are not necessary for convergence in general — configurations with static gates can converge. Adaptive gates are necessary for *robust* convergence (convergence across a wide range of reach values), which is a weaker but still useful claim.

References

[1] W. Ross Ashby. 1956. *An Introduction to Cybernetics.* Chapman & Hall.
[2] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. 2022. Constitutional AI: Harmlessness from AI Feedback. *arXiv preprint arXiv:2212.08073* (2022).
[3] Stafford Beer. 1981. *Brain of the Firm* (2nd ed.). Wiley.
[4] Gérard Berry and Gérard Boudol. 1992. The Chemical Abstract Machine. *Theoretical Computer Science* 96, 1 (1992), 217–248.
[5] Prithviraj Bose. 2025. From Anarchy to Assembly: A Survey of Governance Frameworks for Collaborative LLM Agent Systems. *International Journal of Engineering Research and Emerging Technology (IJERET)* (2025).
[6] Luca Cardelli. 2005. Brane Calculi. In *Computational Methods in Systems Biology (CMSB 2004).* LNCS, Vol. 3082. Springer.
[7] Paul F. Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. 2017. Deep Reinforcement Learning from Human Preferences. In *Advances in Neural Information Processing Systems (NeurIPS).*
[8] European Parliament. 2024. Regulation (EU) 2024/1689 — The Artificial Intelligence Act. Official Journal of the European Union.
[9] Daniel Feitosa et al. 2025. Breaking the Loop: AWARE is the new MAPE-K. *HAL preprint* (2025).
[10] Kevin Anthony Hoff and Masooda Bashir. 2015. Trust in Automation: Integrating Empirical Evidence on Factors That Influence Trust. *Human Factors* 57, 3 (2015), 407–434.
[11] David Kaber. 2025. Systematic Literature Review of Levels of Automation Taxonomy. *International Journal of Human-Computer Interaction* (2025).
[12] Jeffrey O. Kephart and David M. Chess. 2003. The Vision of Autonomic Computing. *IEEE Computer* 36, 1 (2003), 41–50.
[13] Seonghee Kim et al. 2025. Plan-Then-Execute: User Trust and Team Performance with LLM Agents. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25).*
[14] John D. Lee and Katrina A. See. 2004. Trust in Automation: Designing for Appropriate Reliance. *Human Factors* 46, 1 (2004), 50–80.
[15] Martin Leucker and Christian Schallhart. 2009. A Brief Account of Runtime Verification. *Journal of Logic and Algebraic Programming* 78, 5 (2009), 293–303.

[16] Hao Li et al. 2025. Governance Architecture for Autonomous Agent Systems: Threats, Framework, and Engineering Practice. *arXiv preprint arXiv:2603.07191* (2025).

[17] Yushuai Li et al. 2024. Generative AI for Self-Adaptive Systems: State of the Art and Research Roadmap. *ACM Transactions on Autonomous and Adaptive Systems (TAAS)* (2024).

[18] Microsoft. 2026. Agent Governance Toolkit: Open-source runtime security for AI agents. https://github.com/microsoft/agent-governance-toolkit.

[19] NIST. 2023. *AI Risk Management Framework (AI RMF 1.0).* Technical Report NIST AI 100-1. National Institute of Standards and Technology.

[20] Raja Parasuraman, Thomas B. Sheridan, and Christopher D. Wickens. 2000. A Model for Types and Levels of Human Interaction with Automation. *IEEE Transactions on Systems, Man, and Cybernetics—Part A: Systems and Humans* 30, 3 (2000), 286–297.

[21] Gheorghe Păun. 2000. Computing with Membranes. *J. Comput. System Sci.* 61, 1 (2000), 108–143.

[22] Jose Perez Rios. 2025. The Viable System Model and Organizational Pathologies in the Age of AI. *Systems* 13, 9 (2025), 749.

[23] Danny Weyns. 2020. *Introduction to Self-Adaptive Systems: A Contemporary Software Engineering Perspective.* Wiley.

[24] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. In *Proceedings of the Conference on Language Modeling (COLM).*

[25] Ruijie Xiang et al. 2025. AgentGuard: Runtime Verification of AI Agents. *arXiv preprint arXiv:2509.23864* (2025).

[26] Matthew Zurek et al. 2024. SARI: Shared Autonomy across Repeated Interaction. *ACM Transactions on Human-Robot Interaction* (2024).